

卒業研究 現在の進行状況

2026 年 月 日 923044 高宮 悠聖

1. 現在の進行状況

1.1. リバーシ AI

```
1 // 弱AI：合法手の中からランダムに選択
2 List<int>? getWeakAiMove() {
3     // 現在の手番プレイヤーの合法手一覧を取得
4     List<List<int>> validMoves = _generateValidMoves(currentPlayer);
5
6     // 合法手がなければパス
7     if (validMoves.isEmpty()) {
8         return null;
9     }
10
11     validMoves.shuffle(); // 合法手をランダムに並び替え
12
13     return validMoves.first; // 先頭の手を返す
14 }
```

```
1 // 強AI：盤面評価値が最も高いマスを選択
2 List<int>? getStrongAiMove() {
3     // 現在の合法手を取得
4     List<List<int>> validMoves = _generateValidMoves(currentPlayer);
5
6     // 合法手がなければパス
7     if (validMoves.isEmpty()) {
8         return null;
9     }
10
11     List<int> best = validMoves.first; // とりあえず最初の手を候補にする
12     int maxW = -999; // 最大評価値を保存する変数
13
14     // 全ての合法手を調査
15     for (var m in validMoves) {
16         int w = _getStaticWeight(m[0], m[1]); // そのマスの静的評価値を取得
17
18         // 現在の最大値より高ければ更新
19         if (w > maxW) {
20             maxW = w;
21             best = m;
22         }
23     }
24
25     return best; // 最も評価値の高い手を返す
26 }
```

```

1 // 最強AI : NegaScout探索を用いた高性能AI
2 List<int>? getExpertAiMove() {
3     // 現在の合法手を取得
4     List<List<int>> validMoves = _generateValidMoves(currentPlayer);
5
6     // 合法手がなければパス
7     if (validMoves.isEmpty) {
8         return null;
9     }
10
11     // 空きマス数を数える
12     int emptyCells = board
13         .expand((row) => row)
14         .where((cell) => cell == 0)
15         .length;
16
17     bool isSolverMode = emptyCells <= 13; // 終盤なら完全読み切りモードへ移行
18     int targetDepth = isSolverMode ? emptyCells : 10; // 終盤は残り手数まで、
    それ以外は深さ10で探索
19
20     // 置換表が大きくなりすぎた場合は削除
21     if (_transpositionTable.length > 150000) {
22         _transpositionTable.clear();
23     }
24
25     List<int> bestMove = validMoves.first; // 最善手候補
26
27     // 反復深化探索
28     for (int d = 1; d <= targetDepth; d++) {
29         // Alpha-Beta探索の初期値
30         int alpha = -999999999;
31         int beta = 999999999;
32
33         List<int>? currentBestMove; // この深さでの最善手
34
35         // ヒューリスティックを使って手を並び替える
36         _sortMovesWithHeuristics(validMoves, d, currentPlayer);
37
38         // 各合法手を探索
39         for (int i = 0; i < validMoves.length; i++) {
40             var move = validMoves[i];
41
42             // 元の状態を保存
43             int oldHash = _currentHash;
44             int backupPlayer = currentPlayer;
45
46             // マス番号へ変換
47             int idx = move[0] * 8 + move[1];
48
49             // Zobrist Hash更新
50             _currentHash ^= _zobristTable[idx][0];
51             _currentHash ^= _zobristTable[idx][currentPlayer];
52
53             // 仮に着手
54             board[move[0]][move[1]] = currentPlayer;
55
56             // 石を反転し、反転した石を保存
57             List<List<int>> flipped = _flipStonesAndReturn(
58                 move[0],

```

```

59         move[1],
60         currentPlayer,
61     );
62
63     // 次プレイヤーへ変更
64     int nextPlayer = getOpponent(currentPlayer);
65
66     // 手番ハッシュ更新
67     _currentHash ^= _zobristPlayerXor;
68
69     // 相手に合法手があれば手番交代
70     if (hasValidMove(nextPlayer)) {
71         currentPlayer = nextPlayer;
72     }
73
74     int score;
75
76     // 最初の手は通常ウィンドウ探索
77     if (i == 0) {
78         score = -_negascout(d - 1, -beta, -alpha, 60 - d,
isSolverMode);
79     } else {
80         // Null Window Search
81         score = -_negascout(
82             d - 1,
83             -(alpha + 1),
84             -alpha,
85             60 - d,
86             isSolverMode,
87         );
88
89         // Fail-Highなら再探索
90         if (alpha < score && score < beta) {
91             score = -_negascout(d - 1, -beta, -score, 60 - d,
isSolverMode);
92         }
93     }
94
95     // 反転した石を元に戻す
96     for (var stone in flipped) {
97         board[stone[0]][stone[1]] = getOpponent(backupPlayer);
98     }
99
100    // 着手した石を削除
101    board[move[0]][move[1]] = 0;
102
103    // 手番復元
104    currentPlayer = backupPlayer;
105
106    // ハッシュ値復元
107    _currentHash = oldHash;
108
109    // より良い評価値なら更新
110    if (score > alpha) {
111        alpha = score;
112        currentBestMove = move;
113    }
114 }
115

```

```

116         // この深さで最善手が見つければ保存
117         if (currentBestMove != null) {
118             bestMove = currentBestMove;
119         }
120
121         // 勝ち確定レベルなら探索終了
122         if (alpha > 800000) {
123             break;
124         }
125     }
126
127     // 最終的な最善手を返す
128     return bestMove;
129 }
130
131 // 石を反転し、反転した石の位置一覧を返す
132 List<List<int>> _flipStonesAndReturn(int row, int col, int player) {
133     int opponent = getOpponent(player); // 相手プレイヤー番号を取得
134
135     // 反転した石を全て保存するリスト
136     List<List<int>> allFlipped = [];
137
138     // 8方向について調査
139     for (var dir in _directions) {
140         // この方向で反転候補となる石を保存
141         List<List<int>> stonesToFlip = [];
142
143         // 隣接マスから探索開始
144         int r = row + dir[0];
145         int c = col + dir[1];
146
147         // 盤面内である限り探索
148         while (_isInside(r, c)) {
149             // 相手石なら反転候補へ追加
150             if (board[r][c] == opponent) {
151                 stonesToFlip.add([r, c]);
152
153                 // 自分の石に到達した場合
154             } else if (board[r][c] == player) {
155                 // 間にある石を全て反転
156                 for (var stone in stonesToFlip) {
157                     // Zobrist Hash用インデックス
158                     int idx = stone[0] * 8 + stone[1];
159
160                     // ハッシュから相手石状態を除去
161                     _currentHash ^= _zobristTable[idx][opponent];
162
163                     // ハッシュへ自分石状態を追加
164                     _currentHash ^= _zobristTable[idx][player];
165
166                     // 石を反転
167                     board[stone[0]][stone[1]] = player;
168
169                     // 後で戻せるよう保存
170                     allFlipped.add(stone);
171                 }
172
173                 break;
174             } else {

```

```

175         // 空マスなら反転不可
176         break;
177     }
178
179     // 次のマスへ移動
180     r += dir[0];
181     c += dir[1];
182 }
183 }
184
185 return allFlipped; // 反転した石一覧を返す
186 }
187
188 // ♪ NegaScout探索の本体
189 int _negascout(
190     int depth,
191     int alpha,
192     int beta,
193     int ssDepth,
194     bool isSolverMode,
195 ) {
196     int alphaOrig = alpha; // 元のAlpha値を保存
197
198     // 置換表から同一局面を検索
199     TTEEntry? entry = _transpositionTable[_currentHash];
200
201     // 十分な深さで探索済みなら結果を利用
202     if (entry != null && entry.hash == _currentHash && entry.depth >=
depth) {
203         // 完全一致評価値
204         if (entry.flag == 0) {
205             return entry.score;
206
207             // 下限値
208         } else if (entry.flag == 1) {
209             alpha = max(alpha, entry.score);
210
211             // 上限値
212         } else if (entry.flag == 2) {
213             beta = min(beta, entry.score);
214         }
215
216         // 枝刈り成立
217         if (alpha >= beta) {
218             return entry.score;
219         }
220     }
221
222     // 探索終了条件
223     if (depth == 0 || isGameOver()) {
224         // 終盤完全読みモード
225         if (isSolverMode) {
226             return (countStones(currentPlayer) -
countStones(getOpponent(currentPlayer))) *
227                 10000;
228         } else {
229             // 評価関数を利用
230             return _evaluateBoardPattern(currentPlayer);
231         }
232     }
233 }

```

```

232     }
233 }
234
235 // 合法手一覧を取得
236 List<List<int>> validMoves = _generateValidMoves(currentPlayer);
237
238 // パス処理
239 if (validMoves.isEmpty) {
240     int nextPlayer = getOpponent(currentPlayer);
241
242     // 両者とも打てない場合は終了
243     if (!hasValidMove(nextPlayer)) {
244         return (countStones(currentPlayer) - countStones(nextPlayer)) *
10000;
245     }
246
247     currentPlayer = nextPlayer; // 手番を交代して探索継続
248     _currentHash ^= _zobristPlayerXor; // ハッシュ更新
249
250     int score = -_negascout(depth, -beta, -alpha, ssDepth + 1,
isSolverMode);
251
252     // 状態復元
253     _currentHash ^= _zobristPlayerXor;
254     currentPlayer = getOpponent(nextPlayer);
255
256     return score;
257 }
258
259 // 着手順序を最適化
260 _sortNodeMoves(validMoves, ssDepth, entry);
261
262 // この局面での最善手
263 List<int>? bestMoveInThisNode;
264
265 // 全合法手を探索
266 for (int i = 0; i < validMoves.length; i++) {
267     var move = validMoves[i];
268
269     // 元状態保存
270     int oldHash = _currentHash;
271     int backupPlayer = currentPlayer;
272
273     int idx = move[0] * 8 + move[1];
274
275     // 着手によるハッシュ更新
276     _currentHash ^= _zobristTable[idx][0];
277     _currentHash ^= _zobristTable[idx][currentPlayer];
278
279     // 仮着手
280     board[move[0]][move[1]] = currentPlayer;
281
282     // 石反転
283     List<List<int>> flipped = _flipStonesAndReturn(
284         move[0],
285         move[1],
286         currentPlayer,
287     );
288
289     // 相手手番へ変更

```

```

290     int nextPlayer = getOpponent(currentPlayer);
291
292     _currentHash ^= _zobristPlayerXor;
293
294     if (hasValidMove(nextPlayer)) {
295         currentPlayer = nextPlayer;
296     }
297
298     int score;
299
300     // 最初の手は通常探索
301     if (i == 0) {
302         score = -_negascout(
303             depth - 1,
304             -beta,
305             -alpha,
306             ssDepth + 1,
307             isSolverMode,
308         );
309     } else {
310         // Null Window探索
311         score = -_negascout(
312             depth - 1,
313             -(alpha + 1),
314             -alpha,
315             ssDepth + 1,
316             isSolverMode,
317         );
318
319         // 評価値がウィンドウ内なら再探索
320         if (alpha < score && score < beta) {
321             score = -_negascout(
322                 depth - 1,
323                 -beta,
324                 -score,
325                 ssDepth + 1,
326                 isSolverMode,
327             );
328         }
329     }
330
331     // 反転石を元に戻す
332     for (var stone in flipped) {
333         board[stone[0]][stone[1]] = getOpponent(backupPlayer);
334     }
335
336     // 着手を取り消す
337     board[move[0]][move[1]] = 0;
338
339     // 手番復元
340     currentPlayer = backupPlayer;
341
342     // ハッシュ復元
343     _currentHash = oldHash;
344
345     // Beta Cut発生
346     if (score >= beta) {
347         // Killer Move登録
348         if (ssDepth < 60 && _killerMoves[ssDepth][0] != move) {

```

```

349         _killerMoves[ssDepth][1] = _killerMoves[ssDepth][0];
350         _killerMoves[ssDepth][0] = move;
351     }
352
353     // History Heuristic更新
354     _historyTable[move[0]][move[1]] += depth * depth;
355
356     // 置換表へ保存
357     _transpositionTable[_currentHash] = TTEEntry(
358         hash: _currentHash,
359         score: beta,
360         depth: depth,
361         flag: 1,
362         bestMove: move,
363     );
364
365     return beta;
366 }
367
368 // より良い手なら更新
369 if (score > alpha) {
370     alpha = score;
371     bestMoveInThisNode = move;
372 }
373 }
374
375 // 保存する評価種別を決定
376 int flag = (alpha <= alphaOrig) ? 2 : 0;
377
378 // 置換表へ保存
379 _transpositionTable[_currentHash] = TTEEntry(
380     hash: _currentHash,
381     score: alpha,
382     depth: depth,
383     flag: flag,
384     bestMove: bestMoveInThisNode,
385 );
386
387 // 最終評価値を返す
388 return alpha;
389 }
390
391 // 盤面評価関数
392 int _evaluateBoardPattern(int player) {
393     int opponent = getOpponent(player); // 相手プレイヤー番号取得
394     int patternScore = 0; // パターン評価値
395
396     // 上辺評価
397     patternScore += _evaluateEdgePattern(0, 0, 0, 1, 1, 1, player);
398
399     // 下辺評価
400     patternScore += _evaluateEdgePattern(7, 0, 0, 1, 6, 1, player);
401
402     // 左辺評価
403     patternScore += _evaluateEdgePattern(0, 0, 1, 0, 1, 1, player);
404
405     // 右辺評価
406     patternScore += _evaluateEdgePattern(0, 7, 1, 0, 1, 6, player);
407

```



```

408 // 安定石判定結果取得
409 var stableMatrix = _calculateCompleteStableStones();
410
411 int myStables = 0;
412 int oppStables = 0;
413
414 // 安定石数を集計
415 for (int r = 0; r < 8; r++) {
416     for (int c = 0; c < 8; c++) {
417         if (stableMatrix[r][c] == player) {
418             myStables++;
419         } else if (stableMatrix[r][c] == opponent) {
420             oppStables++;
421         }
422     }
423 }
424
425 int myFrontier = 0;
426 int oppFrontier = 0;
427
428 // フロンティア石数を計算
429 for (int r = 0; r < 8; r++) {
430     for (int c = 0; c < 8; c++) {
431         // 空マスは評価対象外
432         if (board[r][c] == 0) {
433             continue;
434         }
435
436         bool isFrontier = false;
437
438         // 周囲8方向を調べる
439         for (var dir in _directions) {
440             int nr = r + dir[0]; // 隣接マスの行
441             int nc = c + dir[1]; // 隣接マスの列
442
443             // 隣に空マスがあればフロンティア石
444             if (_isInside(nr, nc) && board[nr][nc] == 0) {
445                 isFrontier = true;
446                 break;
447             }
448         }
449
450         // フロンティア石を自分・相手別に集計
451         if (isFrontier) {
452             if (board[r][c] == player) {
453                 myFrontier++;
454             } else {
455                 oppFrontier++;
456             }
457         }
458     }
459 }
460
461 int myMobilityEstimate = 0;
462 int oppMobilityEstimate = 0;
463
464 // 行動可能性（可動性）を推定
465 for (int r = 0; r < 8; r++) {
466     for (int c = 0; c < 8; c++) {

```

```

467 // 空マスのみを対象に調査
468 if (board[r][c] == 0) {
469     // 空マスの周囲8方向を確認
470     for (var d in _directions) {
471         int nr = r + d[0]; // 隣接行
472         int nc = c + d[1]; // 隣接列
473
474         if (_isInside(nr, nc)) {
475             // 相手石の隣に空マスがあるほど自分の手候補が増える
476             if (board[nr][nc] == opponent) {
477                 myMobilityEstimate++;
478             }
479
480             // 自分石の隣に空マスがあるほど相手の手候補が増える
481             if (board[nr][nc] == player) {
482                 oppMobilityEstimate++;
483             }
484         }
485     }
486 }
487 }
488 }
489
490 // 可動性評価値を算出
491 int mobilityScore = (myMobilityEstimate - oppMobilityEstimate) * 15;
492
493 // フロンティア石評価値を算出（少ない方が有利）
494 int frontierScore = (oppFrontier - myFrontier) * 18;
495
496 // 安定石評価値を算出（多い方が有利）
497 int stableScore = (myStables - oppStables) * 450;
498
499 // 各評価値を合計して盤面評価値を返す
500 return patternScore + mobilityScore + frontierScore + stableScore;
501 }
502
503 // 辺パターンの評価値を取得
504 int _evaluateEdgePattern(
505     int startR,
506     int startC,
507     int dr,
508     int dc,
509     int x1R,
510     int x1C,
511     int player,
512 ) {
513     int opponent = getOpponent(player); // 相手プレイヤー取得
514     int index = 0; // パターンテーブル参照用インデックス
515
516     // 辺の8マスを3進数としてエンコード
517     for (int i = 0; i < 8; i++) {
518         int cell = board[startR + i * dr][startC + i * dc];
519
520         // 自分=1、相手=2、空=0 に変換
521         int val = (cell == player) ? 1 : ((cell == opponent) ? 2 : 0);
522
523         index += val * _pow3[i];
524     }
525 }

```

```

526 // Xスクエア1を追加
527 int cellX1 = board[x1R][x1C];
528 int valX1 = (cellX1 == player) ? 1 : ((cellX1 == opponent) ? 2 : 0);
529 index += valX1 * _pow3[8];
530
531 // Xスクエア2の座標を計算
532 int x2R = x1R + (dr != 0 ? 5 * dr : 0);
533 int x2C = x1C + (dc != 0 ? 5 * dc : 0);
534
535 // Xスクエア2を追加
536 int cellX2 = board[x2R][x2C];
537 int valX2 = (cellX2 == player) ? 1 : ((cellX2 == opponent) ? 2 : 0);
538 index += valX2 * _pow3[9];
539
540 // 事前計算済みパターン評価値を返す
541 return _edgePatternTable[index];
542 }
543
544 // 完全安定石を計算
545 List<List<int>> _calculateCompleteStableStones() {
546 // 安定石管理用の盤面
547 List<List<int>> stable = List.generate(8, (_) => List.filled(8, 0));
548
549 bool changed = true; // 安定石が増えたかを記録
550
551 // 左上角が埋まっていれば安定石
552 if (board[0][0] != 0) {
553     stable[0][0] = board[0][0];
554 }
555
556 // 右上角が埋まっていれば安定石
557 if (board[0][7] != 0) {
558     stable[0][7] = board[0][7];
559 }
560
561 // 左下角が埋まっていれば安定石
562 if (board[7][0] != 0) {
563     stable[7][0] = board[7][0];
564 }
565
566 // 右下角が埋まっていれば安定石
567 if (board[7][7] != 0) {
568     stable[7][7] = board[7][7];
569 }
570
571 // 安定石が増えなくなるまで繰り返す
572 while (changed) {
573     changed = false;
574
575     for (int r = 0; r < 8; r++) {
576         for (int c = 0; c < 8; c++) {
577             // 空マスまたは既に安定石ならスキップ
578             if (board[r][c] == 0 || stable[r][c] != 0) {
579                 continue;
580             }
581
582             // 横方向の安定判定
583             bool stableHorizontal = _isAxisStable(r, c, 0, 1, stable);
584

```

```

585         // 縦方向の安定判定
586         bool stableVertical = _isAxisStable(r, c, 1, 0, stable);
587
588         // 斜め (＼) 方向の安定判定
589         bool stableDiag1 = _isAxisStable(r, c, 1, 1, stable);
590
591         // 斜め (／) 方向の安定判定
592         bool stableDiag2 = _isAxisStable(r, c, 1, -1, stable);
593
594         // 全方向で安定なら完全安定石とする
595         if (stableHorizontal &&
596             stableVertical &&
597             stableDiag1 &&
598             stableDiag2) {
599             stable[r][c] = board[r][c];
600             changed = true;
601         }
602     }
603 }
604 }
605
606 // 完成した安定石情報を返す
607 return stable;
608 }
609
610 // 指定方向において石が安定石か判定
611 bool _isAxisStable(int r, int c, int dr, int dc, List<List<int>>
stable) {
612     int color = board[r][c]; // 対象石の色
613
614     bool side1Stable = false;
615
616     // 正方向の隣接マスを取得
617     int nr = r + dr;
618     int nc = c + dc;
619
620     // 盤外なら端に接しているため安定
621     if (!_isInside(nr, nc)) {
622         side1Stable = true;
623
624         // 同色の安定石に接している場合も安定
625     } else if (stable[nr][nc] == color) {
626         side1Stable = true;
627     }
628
629     bool side2Stable = false;
630
631     // 逆方向の隣接マスを取得
632     nr = r - dr;
633     nc = c - dc;
634
635     // 盤外なら端に接しているため安定
636     if (!_isInside(nr, nc)) {
637         side2Stable = true;
638
639         // 同色の安定石に接している場合も安定
640     } else if (stable[nr][nc] == color) {
641         side2Stable = true;
642     }

```

```

643
644     // 両方向が安定している場合のみ安定石と判定
645     return side1Stable && side2Stable;
646 }
647
648 // ルート探索時の手順序付け
649 void _sortMovesWithHeuristics(List<List<int>> moves, int depth, int
player) {
650     // History Heuristicと静的位置評価を利用してソート
651     moves.sort((a, b) {
652         // 手Aの評価値
653         int scoreA = _historyTable[a[0]][a[1]] + _getStaticWeight(a[0],
a[1]);
654
655         // 手Bの評価値
656         int scoreB = _historyTable[b[0]][b[1]] + _getStaticWeight(b[0],
b[1]);
657
658         // 高評価の手を先頭へ配置
659         return scoreB.compareTo(scoreA);
660     });
661 }
662
663 // NegaScout探索中の手順序付け
664 void _sortNodeMoves(List<List<int>> moves, int ssDepth, TTEntry? entry)
{
665     // 置換表から最善手候補を取得
666     List<int>? ttMove = entry?.bestMove;
667
668     // Killer Moveを取得
669     List<int>? killer1 = ssDepth < 60 ? _killerMoves[ssDepth][0] : null;
670
671     List<int>? killer2 = ssDepth < 60 ? _killerMoves[ssDepth][1] : null;
672
673     moves.sort((a, b) {
674         int rankA = 0;
675         int rankB = 0;
676
677         // 置換表の最善手を最優先
678         if (ttMove != null && a[0] == ttMove[0] && a[1] == ttMove[1]) {
679             rankA = 100000;
680         }
681
682         if (ttMove != null && b[0] == ttMove[0] && b[1] == ttMove[1]) {
683             rankB = 100000;
684         }
685
686         // 第一Killer Moveを優先
687         if (killer1 != null && a[0] == killer1[0] && a[1] == killer1[1]) {
688             rankA = max(rankA, 50000);
689         }
690
691         if (killer1 != null && b[0] == killer1[0] && b[1] == killer1[1]) {
692             rankB = max(rankB, 50000);
693         }
694
695         // 第二Killer Moveを優先
696         if (killer2 != null && a[0] == killer2[0] && a[1] == killer2[1]) {
697             rankA = max(rankA, 25000);

```

```

698     }
699
700     if (killer2 != null && b[0] == killer2[0] && b[1] == killer2[1]) {
701         rankB = max(rankB, 25000);
702     }
703
704     // ランクが同じ場合はHistory Heuristicで比較
705     if (rankA == rankB) {
706         return _historyTable[b[0]][b[1]].compareTo(_historyTable[a[0]]
707 [a[1]]);
708     }
709
710     // 高ランク順に並べる
711     return rankB.compareTo(rankA);
712 });
713
714 // 盤面位置ごとの固定評価値を取得
715 int _getStaticWeight(int r, int c) {
716     // リバーシ定番の重みテーブル
717     const List<List<int>> weightMap = [
718         // 角は非常に高評価
719         [200, -80, 20, 5, 5, 20, -80, 200],
720
721         // 角の隣 (X・Cマス) は危険なため低評価
722         [-80, -120, -5, -5, -5, -5, -120, -80],
723
724         [20, -5, 15, 3, 3, 15, -5, 20],
725
726         [5, -5, 3, 3, 3, 3, -5, 5],
727
728         [5, -5, 3, 3, 3, 3, -5, 5],
729
730         [20, -5, 15, 3, 3, 15, -5, 20],
731
732         [-80, -120, -5, -5, -5, -5, -120, -80],
733
734         [200, -80, 20, 5, 5, 20, -80, 200],
735     ];
736
737     // 指定座標の評価値を返す
738     return weightMap[r][c];
739 }
740
741 // 3進数エンコード用テーブル
742 static const List<int> _pow3 = [1, 3, 9, 27, 81, 243, 729, 2187, 6561,
743 19683];

```

2. 今後の予定

- ・使用するプログラムの改良